

Process Synchronization

Semaphores, Mutex and the Bounded-Buffer Producer–Consumer Problem

Fundamentals of Operating System Design (FOSD)

DEPSTAR - CHARUSAT

Learning Objectives

By the end of this lecture, students should be able to:

- Explain the purpose and operations of a semaphore.
- Distinguish binary and counting semaphores.
- Explain mutex locks and compare them with binary semaphores.
- Identify the synchronization requirements of the bounded-buffer problem.
- Develop a semaphore-based Producer–Consumer solution.
- Explain the role of `empty`, `full`, and `mutex`.
- Trace producer and consumer execution for a finite buffer.
- Analyze common synchronization errors such as deadlock, race conditions, and incorrect semaphore ordering.

Why Do We Need Semaphores?

A critical-section solution must coordinate concurrent processes.

- Mutual exclusion prevents simultaneous access to shared data.
- Processes may also need to wait for a **condition**.
- Example: a consumer must wait when the buffer is empty.
- Example: a producer must wait when the buffer is full.

Semaphore

A semaphore is a synchronization primitive containing an integer value that is accessed through atomic operations such as **wait()** and **signal()**.

Semaphore = Synchronization + Waiting/Signaling

Semaphore Operations

The two classical atomic operations are:

`wait( $S$ )`

`signal( $S$ )`

wait(S):

$$S = S - 1$$

If the resulting value indicates that the process cannot proceed, the process is blocked.

signal(S):

$$S = S + 1$$

A waiting process may be awakened.

Important

The test and update associated with a semaphore operation must be performed atomically.

Conceptual Semaphore Implementation

A simplified blocking semaphore can be represented as:

wait(S)

- ① Atomically decrement S .
- ② If the process cannot proceed, block it.
- ③ Place it in the semaphore's waiting queue.

signal(S)

- ① Atomically increment/update S .
- ② If processes are waiting, wake one according to the implementation policy.

Conceptual view

wait $\Rightarrow$ request a permit

signal $\Rightarrow$ return a permit

The exact internal implementation is OS- and runtime-dependent.

Binary Semaphore

A **binary semaphore** has two logical states:

$$S \in \{0, 1\}$$

Typical interpretation:

S	Meaning
1	resource available
0	resource unavailable

Typical use:

- Protect a critical section.
- Control access to a single shared resource.
- Provide synchronization between threads/processes.

Example:

$$mutex = 1$$

A process performs:

$wait(mutex) \rightarrow \text{critical section} \rightarrow signal(mutex)$

Counting Semaphore

A **counting semaphore** can represent multiple identical resources.

$$S \geq 0$$

Example: suppose a system has 5 identical resources.

$$S = 5$$

Each successful allocation performs:

$$\textit{wait}(S)$$

Each returned resource performs:

$$\textit{signal}(S)$$

Therefore:

$S = \text{number of currently available resources}$
--

Binary vs Counting Semaphore

Aspect	Binary Semaphore	Counting Semaphore
Value range	0 or 1	Non-negative count
Main purpose	Mutual exclusion / event signaling	Resource counting
Example	One shared resource	10 identical resources
Initialization	Usually 0 or 1	Number of available resources
Multiple permits	No	Yes
Typical use	Lock-like synchronization	Connection/resource pools

Key idea

Binary semaphore answers: “Is the single permit available?”

Counting semaphore answers: “How many permits/resources are available?”

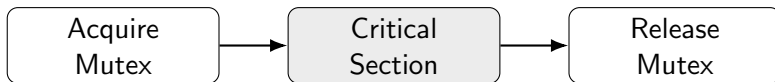
What Is a Mutex?

Mutex = Mutual Exclusion lock

A mutex is specifically designed to protect a critical section.

Typical lifecycle:

lock → critical section → unlock



Ownership

Many mutex implementations have ownership semantics: the thread that locks the mutex is the thread expected to unlock it.

Mutex vs Binary Semaphore

Aspect	Mutex	Binary Semaphore
Primary purpose	Mutual exclusion	Synchronization / mutual exclusion
Ownership	Typically owned by locking thread	Usually no ownership requirement
Operations	lock / unlock	wait / signal
Value/state	Locked / unlocked	0 / 1
Use for signaling	Usually not its primary purpose	Yes
Typical abstraction	Protect critical section	Coordinate events/resources

When to Use Mutex and Semaphore?

Mutex

- Protect shared data.
- One owner at a time.
- Short critical section.
- Example: protect a shared linked list.

Semaphore

- Count available resources.
- Coordinate producer and consumer.
- Signal an event.
- Example: count empty buffer slots.

Rule of thumb

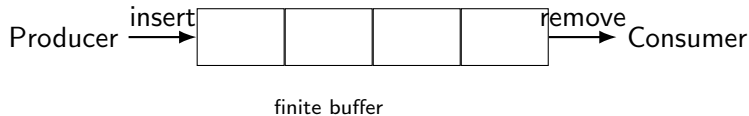
Use a mutex when the primary question is **“Who may enter this critical section?”**

Use a counting semaphore when the primary question is **“How many permits/resources are available?”**

Bounded-Buffer Problem

A bounded buffer is a finite shared queue used by:

- **Producer:** inserts items.
- **Consumer:** removes items.



Two constraints:

Producer must not insert into a full buffer

Consumer must not remove from an empty buffer

Bounded Buffer: Why Synchronization Is Required

Suppose the buffer contains 3 slots:

$$[B_0] \ [B_1] \ [B_2]$$

Potential problems:

- Producer and consumer may update buffer indices simultaneously.
- Two producers may write to the same slot.
- Two consumers may remove the same item.
- Producer may overwrite an item that has not been consumed.
- Consumer may read an empty slot.

Therefore we need:

Mutual exclusion + empty/full synchronization

Three Semaphores for Producer–Consumer

For buffer size N , initialize:

$$\text{mutex} = 1$$

$$\text{empty} = N$$

$$\text{full} = 0$$

Meaning:

Semaphore	Initial value	Meaning
mutex	1	Only one process manipulates buffer state at a time
empty	N	Number of empty slots
full	0	Number of occupied slots

Invariant:

$$\text{empty} + \text{full} = N$$

This invariant is maintained as items are produced and consumed.

Producer: Required Sequence

A producer must:

- 1 Wait for an empty slot.
- 2 Acquire exclusive access to the buffer.
- 3 Insert the item.
- 4 Release exclusive access.
- 5 Signal that one more item is available.

Sequence:

$$wait(empty) \rightarrow wait(mutex) \rightarrow insert \rightarrow signal(mutex) \rightarrow signal(full)$$

Reason

The producer cannot insert unless an empty slot exists.

Consumer: Required Sequence

A consumer must:

- 1 Wait for an available item.
- 2 Acquire exclusive access to the buffer.
- 3 Remove the item.
- 4 Release exclusive access.
- 5 Signal that one more slot is empty.

Sequence:

$$wait(full) \rightarrow wait(mutex) \rightarrow remove \rightarrow signal(mutex) \rightarrow signal(empty)$$

Reason

The consumer cannot remove an item unless the buffer contains at least one item.

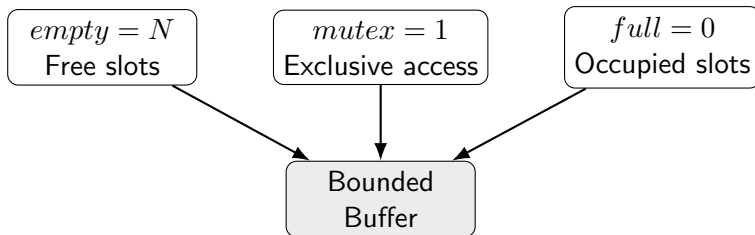
Producer Algorithm

```
do {  
    item = produce_item();  
  
    wait(empty);          // reserves an empty slot  
    wait(mutex);          // protects the shared buffer  
  
    insert_item(item);  
  
    signal(mutex);  
    signal(full);         // announces that a new item is available  
  
} while (true);
```

Consumer Algorithm

```
do {  
    wait(full);           // ensures that an item exists.  
    wait(mutex);         // protects the shared buffer.  
  
    item = remove_item();  
  
    signal(mutex);  
    signal(empty);        //announces that a buffer slot is now free.  
  
    consume_item(item);  
  
} while (true);
```

Why Are Three Semaphores Needed?



- $empty$ controls **capacity**.
- $full$ controls **availability of items**.
- $mutex$ controls **exclusive access**.

Worked Example: Buffer Size = 5

Initially:

$$empty = 5, \quad full = 0, \quad mutex = 1$$

Producer inserts one item:

$$wait(empty) : \quad empty = 4$$

$$wait(mutex) : \quad mutex = 0$$

Insert item.

$$signal(mutex) : \quad mutex = 1$$

$$signal(full) : \quad full = 1$$

Final state:

$empty = 4, \quad full = 1, \quad mutex = 1$
--

Worked Example: Full Buffer

Suppose:

$$N = 5, \quad \textit{empty} = 0, \quad \textit{full} = 5$$

A producer executes:

$$\textit{wait}(\textit{empty})$$

Because no empty slot exists, the producer must wait/block in a blocking semaphore implementation.

Meanwhile, a consumer can execute:

$$\textit{wait}(\textit{full})$$

After consuming:

$$\textit{full} = 4$$

and:

$$\textit{signal}(\textit{empty}) \Rightarrow \textit{empty} = 1$$

Now a producer can proceed.

Worked Example: Empty Buffer

Suppose:

$$empty = 5, \quad full = 0$$

A consumer executes:

$$wait(full)$$

Since:

$$full = 0$$

the consumer must wait/block.

A producer then inserts an item:

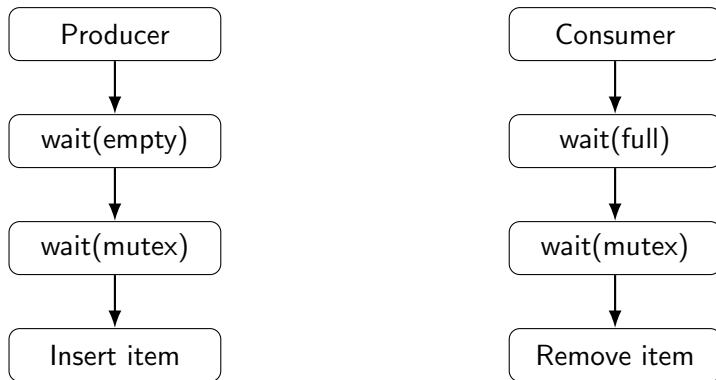
$$signal(full)$$

Now:

$$full = 1$$

The waiting consumer can be awakened and proceed.

Producer-Consumer Execution Flow



Both producer and consumer use the same *mutex* to protect the shared buffer.

Semaphore Ordering Is Important

Correct producer order:

$$\text{wait}(\text{empty}) \rightarrow \text{wait}(\text{mutex})$$

Correct consumer order:

$$\text{wait}(\text{full}) \rightarrow \text{wait}(\text{mutex})$$

Why?

The process should first establish that the required resource/condition exists and then acquire the buffer lock.

Potential problem

Incorrect ordering can create deadlock.

For example, if a producer acquires *mutex* before waiting for *empty*, it could hold the mutex while the buffer is full, preventing a consumer from acquiring the mutex to remove an item.

Deadlock Example from Wrong Ordering

Assume:

$$empty = 0$$

and the buffer is full.

Incorrect producer:

$wait(mutex)$

$wait(empty)$

The producer obtains *mutex* and then blocks on *empty*.

Now a consumer needs:

$wait(full) \rightarrow wait(mutex)$

But the consumer cannot obtain *mutex*.

Therefore:

Producer waits for empty and Consumer waits for mutex

This creates a circular wait and can lead to deadlock.

Why mutex Must Protect the Buffer

Suppose two producers execute concurrently.

Without mutex:

P_1 : choose slot i

P_2 : choose slot i

Both may update the same shared buffer position.

Similarly, two consumers may select the same item.

With:

$wait(mutex)$

only one process manipulates the shared buffer state at a time.

Separation of responsibilities

$empty/full \Rightarrow$ condition synchronization

$mutex \Rightarrow$ mutual exclusion

Bounded Buffer Invariant

For a buffer of capacity N :

$$\boxed{empty + full = N}$$

Initially:

$$empty = N, \quad full = 0$$

After production:

$$empty \downarrow, \quad full \uparrow$$

After consumption:

$$empty \uparrow, \quad full \downarrow$$

Therefore the number of occupied and free slots always accounts for the entire buffer.

Useful debugging technique

If a semaphore-based implementation violates this invariant, inspect the corresponding `wait()` and `signal()` operations.

Producer–Consumer: State Table

For $N = 5$:

Operation	empty	full	Meaning
Initial	5	0	Buffer empty
Produce 1	4	1	One item available
Produce 2	3	2	Two items available
Consume 1	4	1	One item remains
Consume 2	5	0	Buffer empty again

At every point:

$$empty + full = 5$$

Real-World Analogy: Print Queue

Consider a printer with a finite print queue.

- **Producers:** applications submitting print jobs.
- **Consumer:** printer service processing jobs.
- **Buffer:** finite print queue.

empty = available queue slots

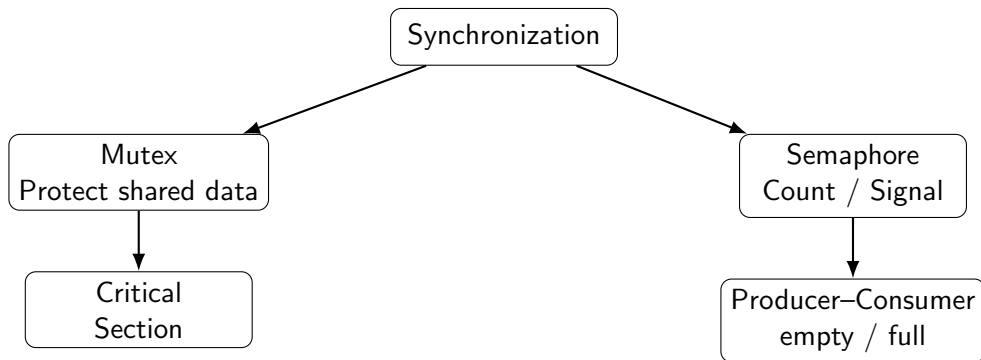
full = queued print jobs

mutex = exclusive access to queue data structures

If the queue is full, a new producer must wait.

If the queue is empty, the printer service has no job to process.

Semaphore vs Mutex: Practical View



Common Errors in Producer–Consumer

- ❶ Forgetting the *mutex* semaphore.
- ❷ Forgetting *empty* or *full*.
- ❸ Initializing *full* to N instead of 0.
- ❹ Initializing *empty* to 0 instead of N .
- ❺ Calling `signal(empty)` before an item is actually removed.
- ❻ Calling `signal(full)` before an item is actually inserted.
- ❼ Acquiring *mutex* before waiting for *empty/full*.
- ❽ Using non-atomic semaphore operations.

Exam tip

Always write the purpose and initial value of every semaphore before writing the algorithm.

HOTS Question 1

A bounded buffer has capacity 10.

Initially:

$$empty = 10, \quad full = 0$$

Five items are produced and two are consumed.

Find:

$$empty, \quad full$$

Solution

After five productions:

$$empty = 5, \quad full = 5$$

After two consumptions:

$$empty = 7, \quad full = 3$$

Check:

$$7 + 3 = 10$$

HOTS Question 2

A buffer has capacity 5 and is currently full.

A producer executes:

wait(mutex)

before:

wait(empty)

Analyze what can happen if the producer obtains the mutex.

Expected analysis

The producer may block on *empty* = 0 while holding *mutex*. A consumer that needs the same mutex cannot enter the critical section to remove an item and signal *empty*. This can result in deadlock.

HOTS Question 3: Design Decision

A system has:

100 identical database connections

Multiple worker threads need to acquire a connection.

Which synchronization abstraction is conceptually appropriate?

- ① Mutex only
- ② Counting semaphore initialized to 100
- ③ Binary semaphore initialized to 100
- ④ Peterson's solution

Answer

Counting semaphore initialized to 100

Each successful `wait()` consumes one connection permit, and each `signal()` returns one.

Exam Question: Identify the Semaphores

For the bounded-buffer problem with capacity N :

$mutex = ?$

$empty = ?$

$full = ?$

Complete the table:

Semaphore	Initial value	Purpose
$mutex$	?	?
$empty$	?	?
$full$	?	?

Answer

$mutex = 1,$ $empty = N,$ $full = 0$

Summary

- A **semaphore** is a synch. primitive accessed through atomic wait/signal operations.
- A **binary semaphore** has two logical states and can provide mutual exclusion or signaling.
- A **counting semaphore** represents multiple permits/resources.
- A **mutex** is primarily used to protect a CS and commonly has ownership semantics.
- The bounded-buffer problem requires both:

Mutual Exclusion + Condition Synchronization

- Standard solution:

$mutex = 1, \quad empty = N, \quad full = 0$

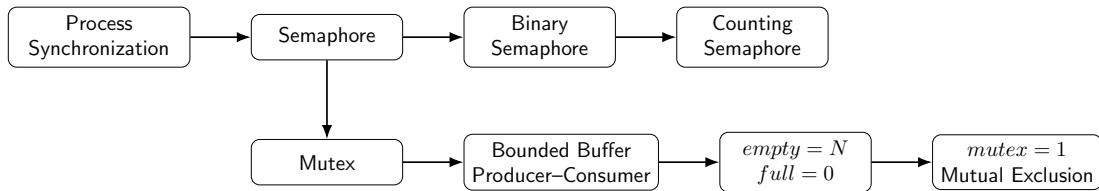
- Producer:

$wait(empty) \rightarrow wait(mutex) \rightarrow insert \rightarrow signal(mutex) \rightarrow signal(full)$

- Consumer:

$wait(full) \rightarrow wait(mutex) \rightarrow remove \rightarrow signal(mutex) \rightarrow signal(empty)$

Concept Map



Monitor and Condition Variables

CEUC301 - Fundamentals of Operating System Design

Prashant Jadav

DEPSTAR

Learning Objectives

By the end of this lecture, students should be able to:

- Explain the concept of a monitor.
- Identify the components of a monitor.
- Explain mutual exclusion provided by monitors.
- Explain condition variables and their purpose.
- Explain `wait()`, `signal()` and notification.
- Distinguish busy waiting from blocking synchronization.
- Compare monitors with semaphores.
- Implement monitor-based synchronization in Java.
- Analyze the Producer–Consumer problem using Java monitors.

Why Do We Need Monitors?

Concurrent threads often share resources such as:

- Shared variables
- Buffers
- Queues
- Files
- Database records
- Device resources

Why Do We Need Monitors?

Without synchronization, concurrent access can produce a:

Race Condition

A situation in which the result depends on the timing or interleaving of concurrent thread execution.

We need mechanisms that provide:

- ① Mutual exclusion
- ② Safe access to shared data
- ③ Condition-based waiting
- ④ Notification of waiting threads

Monitor: Definition

Monitor

A monitor is a high-level synchronization construct that encapsulates shared data, procedures operating on that data, and synchronization mechanisms.

A monitor provides:

- **Mutual exclusion**
- **Condition synchronization**

Only one thread can execute a monitor procedure at a time.

$$\text{Monitor} = \text{Shared Data} + \text{Procedures} + \text{Synchronization}$$

Basic Structure of a Monitor

Conceptually:

MONITOR
Shared Data
Procedure 1 Procedure 2 Procedure 3
Condition Variables

- Shared data is encapsulated inside the monitor.
- Threads access data through monitor procedures.
- Mutual exclusion is automatically enforced.
- Condition variables allow threads to wait for conditions.

Monitor Provides Mutual Exclusion

Suppose two threads execute:

```
1 synchronized void increment() {  
2     count++;  
3 }
```

For the same object:

- ① Thread T1 acquires the object's monitor.
- ② T1 executes `increment()`.
- ③ T2 cannot enter another synchronized method of the same object.
- ④ T1 exits and releases the monitor.
- ⑤ T2 can acquire the monitor.

Therefore:

At most one thread executes protected code

Java Object Monitor

Every Java object can be associated with an intrinsic lock.

Example:

```
1 synchronized (obj) {  
2     // critical section  
3 }
```

The executing thread must acquire the monitor associated with obj.

Conceptually:

Thread → Acquire Monitor → Critical Section → Release Monitor

If the monitor is already owned:

Thread → BLOCKED

Condition Variables in a Monitor

A monitor uses **condition variables** to allow a process or thread to wait for a particular condition.

Two fundamental operations are associated with a condition variable:

Operation	Purpose
<code>cwait(condition)</code>	Suspend the calling process until the specified condition becomes true.
<code>csignal(condition)</code>	Signal a process waiting on the specified condition.

Condition Variable + `cwait()` + `csignal()`

provides condition synchronization.

cwait Operation

The operation:

cwait(condition)

causes the calling process/thread to wait for a condition.

Typical sequence:

- 1 Process enters the monitor.
- 2 It checks a condition.
- 3 Condition is not satisfied.
- 4 Process executes `cwait(condition)`.
- 5 Process is suspended.
- 6 The monitor becomes available to another process.

Example:

```
1 while (bufferEmpty) {  
2     cwait(notEmpty);  
3 }
```

csignal Operation

The operation:

`csignal(condition)`

signals a process waiting on the specified condition.

For example, after a producer inserts an item:

```
1 insert(item);  
2  
3 csignal(notEmpty);
```

The producer changes the buffer state:

Buffer Empty → Buffer Not Empty

and signals a waiting consumer.

Producer–Consumer with cwait and csignal

A conceptual monitor solution:

```
1 monitor BoundedBuffer {  
2     condition notFull;  
3     condition notEmpty;  
4  
5     procedure insert(item) {  
6         while (bufferFull) {  
7             cwait(notFull);  
8         }  
9         insert item into buffer;  
10        csignal(notEmpty);  
11    }  
12    procedure remove() {  
13        while (bufferEmpty) {  
14            cwait(notEmpty);  
15        }  
16        remove item from buffer;  
17        csignal(notFull);  
18    }  
19 }
```

Producer–Consumer Condition Variables

The producer waits when:

Buffer is full

Therefore:

```
cwait(notFull)
```

After consuming an item:

Buffer Full → Space Available

Consumer signals:

```
csignal(notFull)
```

The consumer waits when:

Monitor Operations: Complete Picture

A monitor provides two types of synchronization.

1. Mutual exclusion

Only one process executes inside the monitor

2. Condition synchronization

`cwait(condition)`

and

`csignal(condition)`

Therefore:

Monitor = Mutual Exclusion + Condition Synchronization

cwait vs csignal

cwait	csignal
Makes process wait	Wakes/signals a waiting process
Process becomes suspended	Waiting process becomes eligible to run
Used when condition is false	Used after condition changes

Example:

```
cwait(notEmpty)
```

waits for an item.

```
csignal(notEmpty)
```

signals that an item is available.

Classical Monitor vs Java

Classical Monitor	Java Intrinsic Monitor	Java Condition API
Monitor	Object monitor	Lock
Mutual exclusion	synchronized	lock()
cwait()	wait()	await()
csignal()	notify()	signal()
Signal all	notifyAll()	signalAll()

Java Explicit Condition Variables

Java provides explicit condition variables through:

```
1 import java.util.concurrent.locks.*;  
2  
3 Lock lock = new ReentrantLock();  
4  
5 Condition notEmpty =  
6     lock.newCondition();  
7  
8 Condition notFull =  
9     lock.newCondition();
```

The mapping is:

`cwait(notEmpty) → notEmpty.await()`

`csignal(notEmpty) → notEmpty.signal()`

Java Condition Variable Example

```
1 lock.lock();
2 try {
3     while (count == 0) {
4         notEmpty.await();
5     }
6     int item = buffer[out];
7
8     out = (out + 1) % buffer.length;
9     count--;
10
11     notFull.signal();
12 } finally {
13     lock.unlock();
14 }
```

`cwait(notEmpty)` $\longleftrightarrow$ `notEmpty.await()`

`csignal(notFull)` $\longleftrightarrow$ `notFull.signal()`

What Is a Condition Variable?

A condition variable allows a thread to wait until a particular condition becomes true.

It provides:

- Efficient waiting
- Blocking instead of busy waiting
- Notification when the shared state changes

Typical monitor operations are:

Operation	Purpose
wait	Block until a condition may have changed
signal	Wake one waiting thread
signalAll	Wake all waiting threads

In Java:

- `wait()`
- `notify()`
- `notifyAll()`

Why Do We Need Condition Variables?

Consider a bounded buffer.

The producer wants to insert an item, but the Buffer is full:

The producer cannot proceed.

One approach is busy waiting:

```
1 while (bufferFull) {  
2     // continuously check  
3 }
```

This wastes CPU time. A monitor provides blocking:

```
1 while (bufferFull) {  
2     wait();  
3 }
```

The thread sleeps until the condition changes.

Condition variables provide efficient waiting

Busy Waiting vs Blocking

	Busy Waiting	Blocking
CPU usage	High	Low while waiting
Waiting mechanism	Repeated checking	Thread blocks
Example	Repeated condition test	<code>wait()</code>
CPU wasted?	Yes	No while blocked
Typical use	Very short waits	Longer waits

Important

A while loop does not automatically mean busy waiting.
`while(condition){ wait(); }` is blocking synchronization.

Condition Variable: Basic Pattern in Java

```
1 synchronized void operation() throws InterruptedException {  
2     while (!condition) {  
3         wait();  
4     }  
5     // condition is true  
6     // perform operation  
7     notifyAll();  
8 }
```

Execution sequence:

- 1 Acquire monitor.
- 2 Check condition.
- 3 If false, call wait().
- 4 Release monitor and block.
- 5 Another thread changes the condition.
- 6 Notification occurs.
- 7 Waiting thread reacquires the monitor.
- 8 Condition is checked again.

Why Use while Instead of if?

Problem

A thread should not assume that the condition is still true after it wakes up.

Preferred pattern:

- Check the condition.
- Wait if the condition is false.
- Wake up.
- Check the condition again.

Therefore:

```
while(condition){ wait(); }
```

is preferred over:

```
if(condition){ wait(); }
```

Java Monitor Mechanisms

Mechanism	Purpose
synchronized	Mutual exclusion
wait()	Release monitor and wait
notify()	Wake one waiting thread
notifyAll()	Wake all waiting threads

Important

A thread must own the object's monitor before calling `wait()`, `notify()`, or `notifyAll()`.

This normally means they are called inside:

- a `synchronized` method, or
- a `synchronized(obj)` block.

Producer–Consumer Problem

Consider a bounded buffer with capacity N .

- Producer adds items.
- Consumer removes items.
- Producer must wait when buffer is full.
- Consumer must wait when buffer is empty.

Two logical conditions exist:

notFull

and

notEmpty

The monitor protects the shared buffer and its state.

Producer Using a Monitor

The producer executes:

```
1 public synchronized void put(int item)
2     throws InterruptedException {
3
4     while (count == buffer.length) {
5         wait();
6     }
7
8     buffer[in] = item;
9     in = (in + 1) % buffer.length;
10    count++;
11
12    notifyAll();
13 }
```

When the buffer is full:

Producer → wait() → Monitor released → Producer blocked

Consumer Using a Monitor

The consumer executes:

```
1 public synchronized int get() throws InterruptedException {  
2  
3     while (count == 0) {  
4         wait();  
5     }  
6  
7     int item = buffer[out];  
8  
9     out = (out + 1) % buffer.length;  
10    count--;  
11  
12    notifyAll();  
13  
14    return item;  
15 }
```

When the buffer is empty:

Consumer → wait() → Monitor released → Consumer blocked

Java Bounded Buffer

```
1 class Buffer {
2
3     private final int[] buffer = new int[5];
4
5     private int count = 0;
6     private int in = 0;
7     private int out = 0;
8
9     public synchronized void put(int item) throws InterruptedException {
10
11         while (count == buffer.length) {
12             wait();
13         }
14
15         buffer[in] = item;
16         in = (in + 1) % buffer.length;
17         count++;
18
19         System.out.println("Produced: " + item);
20
21         notifyAll();
22     }
```

Java Bounded Buffer

```
1
2 public synchronized int get()
3     throws InterruptedException {
4
5     while (count == 0) {
6         wait();
7     }
8
9     int item = buffer[out];
10
11     out = (out + 1) % buffer.length;
12     count--;
13
14     System.out.println("Consumed: " + item);
15
16     notifyAll();
17
18     return item;
19 }
20 }
```

Producer Thread

```
1 class Producer implements Runnable {  
2  
3     private final Buffer buffer;  
4  
5     Producer(Buffer buffer) {  
6         this.buffer = buffer;  
7     }  
8  
9     public void run() {  
10         try {  
11             for (int i = 1; i <= 10; i++) {  
12                 buffer.put(i);  
13                 Thread.sleep(100);  
14             }  
15  
16             } catch (InterruptedException e) {  
17                 Thread.currentThread().interrupt();  
18             }  
19         }  
20     }
```

Consumer Thread

```
1 class Consumer implements Runnable {
2
3     private final Buffer buffer;
4
5     Consumer(Buffer buffer) {
6         this.buffer = buffer;
7     }
8
9     public void run() {
10
11         try {
12             for (int i = 1; i <= 10; i++) {
13                 buffer.get();
14                 Thread.sleep(150);
15             }
16         } catch (InterruptedException e) {
17             Thread.currentThread().interrupt();
18         }
19     }
20 }
```

Main Program

```
1 public class MonitorDemo {  
2  
3     public static void main(String[] args) {  
4  
5         Buffer buffer = new Buffer();  
6  
7         Thread producer = new Thread(new Producer(buffer));  
8  
9         Thread consumer = new Thread(new Consumer(buffer));  
10  
11         producer.start();  
12         consumer.start();  
13     }  
14 }
```

Important:

Producer and Consumer share the SAME Buffer object

Therefore they synchronize using the same monitor.

When Buffer Is Empty

Suppose:

count = 0

Consumer executes:

- 1 Acquires the buffer monitor.
- 2 Checks `count == 0`.
- 3 Calls `wait()`.
- 4 Releases the monitor.
- 5 Becomes blocked.

The producer can now acquire the monitor.

Key Point

The consumer is not continuously executing the condition test. It is blocked and therefore does not consume CPU while waiting.

Producer Adds an Item

Producer performs:

- ① Acquire monitor.
- ② Insert item.
- ③ Increment count.
- ④ Call `notifyAll()`.

A waiting consumer becomes eligible to continue.

However:

`notifyAll()  $\neq$  immediate lock transfer`

The consumer must first reacquire the monitor.

Then it rechecks:

```
while(count == 0)
```

notify vs notifyAll

Method	Meaning
<code>notify()</code>	Wakes one thread waiting on this object's monitor.
<code>notifyAll()</code>	Wakes all threads waiting on this object's monitor.

After being awakened:

Waiting → Runnable → Acquire Monitor → Continue

The awakened thread does not automatically own the monitor.

Monitor vs Semaphore

	Monitor	Semaphore
Abstraction	High-level	Synchronization primitive
Mutual exclusion	Built in	Must be designed correctly
Shared data	Encapsulated	Not necessarily encapsulated
Waiting	Condition variables	Semaphore operations
Programming safety	Generally easier	Easier to misuse
Java support	synchronized	Semaphore class

Monitor vs Busy Waiting

Busy waiting:

```
while(bufferEmpty){ }
```

The processor repeatedly executes the loop.

Monitor-based waiting:

```
while(bufferEmpty){ wait(); }
```

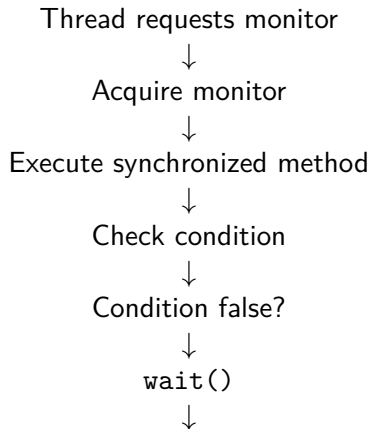
The thread:

Blocks → Consumes no CPU while waiting → Wakes → Rechecks condition

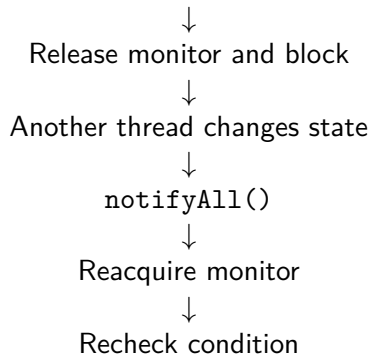
Conclusion

Monitor-based condition waiting avoids CPU-consuming busy waiting.

Monitor Execution Model



Monitor Execution Model



Advantages of Monitors

- High-level synchronization abstraction.
- Automatic mutual exclusion.
- Shared data can be encapsulated.
- Supports condition-based waiting.
- Avoids unnecessary busy waiting.
- Easier to reason about than raw semaphore operations.
- Java provides built-in monitor support.

Common Programming Errors

- Calling `wait()` outside synchronized code.
- Calling `notify()` without owning the monitor.
- Using `if` instead of `while`.
- Synchronizing on different objects.
- Forgetting to notify waiting threads.
- Assuming `notify()` immediately transfers the lock.

Correct pattern:

```
while(condition){ wait(); }
```

Key Takeaways

- A monitor is a high-level synchronization abstraction.
- It combines shared data, procedures and synchronization.
- Mutual exclusion is automatically enforced.
- Condition variables provide condition-based waiting.
- Java supports monitors using object monitors and `synchronized`.
- Java's traditional monitor operations are `wait()`, `notify()`, and `notifyAll()`.
- `wait()` releases the monitor while waiting.
- Waiting with `wait()` is not busy waiting.
- Conditions should normally be checked using a `while` loop.
- Producer–Consumer is a classic application.

Discussion and Examination Questions

- 1 What is a monitor? Explain its components.
- 2 Why is a monitor considered a high-level synchronization mechanism?
- 3 Differentiate busy waiting and blocking.
- 4 What happens to the monitor lock when `wait()` is called?
- 5 Why should `wait()` normally be used inside a `while` loop?
- 6 Differentiate `notify()` and `notifyAll()`.
- 7 Explain how Java implements monitor-based synchronization.
- 8 Design a monitor solution for the Producer–Consumer problem.

Readers–Writers Problem

Semaphore-Based Solutions: Readers Priority and Writers Priority

Prashant Jadav

Department of Computer Engineering
DEPSTAR – CHARUSAT

Readers–Writers Problem

Problem

A shared data object (database, file, memory structure, etc.) is accessed by multiple concurrent processes:

- **Reader:** only reads the shared data.
- **Writer:** modifies the shared data.

Synchronization requirement

- Multiple readers may read simultaneously.
- A writer requires **exclusive access**.
- A reader cannot access the data while a writer is writing.

Real-world example: Many users can view a database record simultaneously, but an update operation must have exclusive access.

Core Idea: Semaphore-Based Synchronization

Let the shared database/resource be protected by semaphore `wrt`.

- `wrt = 1`: database is available.
- `wrt = 0`: a writer is using it, or readers collectively hold it.
- `mutex`: protects the variable `readcount`.

Reader grouping

The first reader performs:

`wait(wrt)`

and the last reader performs:

`signal(wrt)`

Therefore, several readers can execute concurrently while writers are excluded.

Core Idea: Semaphore-Based Synchronization

Important issue: starvation

The basic solution must decide who gets preference when both readers and writers are waiting:

- **Readers priority** → writers may starve.
- **Writers priority** → readers may wait.

Readers Priority: Reader Pseudocode

Initialization:

`mutex = 1,      wrt = 1,      readcount = 0`

Reader Pseudocode

```
wait(mutex);  
readcount++;
```

```
if (readcount == 1)  
    wait(wrt);
```

```
signal(mutex);
```

```
READ_DATABASE();
```

```
wait(mutex);  
readcount--;
```

```
if (readcount == 0)  
    signal(wrt);
```

```
signal(mutex);
```

Readers Priority: Writer Pseudocode

Writer Pseudocode

```
wait(wrt);
```

```
WRITE_DATABASE();
```

```
signal(wrt);
```

Key idea

The writer requires exclusive access to the shared database.

Readers-Priority: Execution Logic

Event	Reader	Writer
First reader	Acquires wrt	Must wait
Additional readers	Enter concurrently	Must wait
Last reader exits	Releases wrt	May enter
Writer active	Must wait	Exclusive access

Example sequence:

$$R_1 \rightarrow R_2 \rightarrow W_1 \rightarrow R_3$$

If R_3 arrives while R_1 and R_2 are still reading, it can join the reader group. W_1 waits until the last reader exits.

When useful?

Read-heavy systems where delaying readers is more costly than delaying occasional writes.

Writers Priority: Initialization

Semaphores and variables

```
resource = 1      readTry = 1  
rmutex = 1        wmutex = 1  
readcount = 0    writecount = 0
```

Purpose

- resource: controls exclusive database access.
- readTry: controls entry of new readers.
- rmutex: protects readcount.
- wmutex: protects writecount.

Writers Priority: Reader Pseudocode

```
wait(readTry);  
wait(rmutex);  
readcount++;  
  
if (readcount == 1)  
    wait(resource);  
  
signal(rmutex);  
signal(readTry);  
  
READ_DATABASE();  
  
wait(rmutex);  
readcount--;  
  
if (readcount == 0)  
    signal(resource);  
  
signal(rmutex);
```

Writers Priority: Writer Pseudocode

```
wait(wmutex);  
writecount++;  
  
if (writecount == 1)  
    wait(readTry);  
  
signal(wmutex);  
  
wait(resource);  
WRITE_DATABASE();  
signal(resource);  
  
wait(wmutex);  
writecount--;  
  
if (writecount == 0)  
    signal(readTry);  
  
signal(wmutex);
```

Writers-Priority: Why readTry Is Needed

Key mechanism

readTry acts as a **gate for incoming readers**.

- 1 First waiting writer increments writecount.
- 2 It executes wait(readTry).
- 3 New readers cannot pass the readTry gate.
- 4 Existing readers are allowed to finish.
- 5 Waiting writers obtain resource one at a time.
- 6 When the last writer leaves, readTry is released.
- 7 Readers can enter again.

Trade-off

Writer priority reduces writer starvation, but if writers continuously arrive, readers may experience starvation or very long waiting times.

Semaphore Roles in Both Solutions

Semaphore	Readers Priority	Writers Priority
wrt/resource	Controls exclusive database access	Controls exclusive database access
mutex/rmutex	Protects readcount	Protects readcount
wmutex	Not required	Protects writecount
readTry	Not required	Gate that prevents new readers when writers are waiting

Core distinction

Readers priority: first reader locks the resource

Writers priority: first waiting writer closes the reader gate

Readers Priority vs Writers Priority

Aspect	Readers Priority	Writers Priority
Preference	Readers	Writers
Concurrent readers	Yes	Yes
Concurrent writers	No	No
Writer starvation	Possible	Reduced
Reader starvation	Less likely	Possible
Main mechanism	<code>mutex + wrt</code>	<code>resource + readTry + mutexes</code>
Best suited for	Read-heavy workloads	Systems where writes must not wait behind new readers
Main trade-off	Writers can wait indefinitely	Readers can wait behind writers

Readers–Writers vs Producer–Consumer

Aspect	Readers–Writers	Producer–Consumer
Shared object	Database/file/data structure	Bounded buffer/queue
Operations	Read vs. modify	Produce vs. consume
Concurrent read-ers/consumers	Multiple readers allowed	Multiple consumers may be allowed
Writer/producer	Exclusive writer	Producer adds an item
Main constraint	Writers need exclusive access	Buffer must not overflow/underflow
Typical semaphores	mutex, wrt	empty, full, mutex
Main issue	Fairness/starvation	Buffer synchronization
Typical policy	Reader or writer priority	Producer/consumer coordination

Producer–Consumer: Contrast in Semaphore Logic

Bounded-buffer problem

For a buffer of size N :

$$\text{empty} = N, \quad \text{full} = 0, \quad \text{mutex} = 1$$

Producer $\text{wait}(\text{empty}) \rightarrow \text{wait}(\text{mutex}) \rightarrow \text{insert} \rightarrow \text{signal}(\text{mutex}) \rightarrow \text{signal}(\text{full})$

Consumer $\text{wait}(\text{full}) \rightarrow \text{wait}(\text{mutex}) \rightarrow \text{remove} \rightarrow \text{signal}(\text{mutex}) \rightarrow \text{signal}(\text{empty})$

Key difference

- **Readers–Writers:** synchronization is mainly about **access compatibility and exclusivity**.
- **Producer–Consumer:** synchronization is mainly about **buffer capacity and item availability**.

Key Takeaways

- 1 Readers can safely share a resource; writers require exclusive access.
- 2 The first/last-reader technique allows concurrent readers.
- 3 **Readers-priority** favors throughput for readers but can starve writers.
- 4 **Writers-priority** prevents new readers from bypassing waiting writers, but readers may experience long waits.
- 5 `readTry` is the key gate used in the common writers-priority solution.
- 6 Producer–Consumer solves a different synchronization problem: coordinating access to a bounded shared buffer.

Exam perspective

Be able to identify:

- Which semaphore protects the shared resource?
- Which variable counts active readers/writers?
- Which process can starve?
- Why does the writers-priority solution need `readTry`?